

GUÍA DE LABORATORIO N.º 7

Autenticación con JWT: Access Tokens y Refresh Tokens

NestJS 11 · PostgreSQL · TypeORM

Asignatura	Seguridad de Software
Sigla	INF-781
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Unidad temática	Autenticación y gestión de sesiones
Gestión	I - 2026

1. Objetivos

Objetivo general

Implementar un sistema de autenticación stateless basado en JSON Web Tokens (JWT) sobre una API construida con NestJS y PostgreSQL, empleando un esquema de access token y refresh token para gestionar el acceso y la renovación de credenciales.

Objetivos específicos

- Comprender la estructura, la firma y el ciclo de vida de un JWT.
- Distinguir el rol y las características del access token frente al refresh token.
- Crear un proyecto NestJS desde cero y conectarlo a PostgreSQL mediante TypeORM.
- Implementar el registro y el inicio de sesión con hashing seguro de contraseñas (argon2).
- Emitir y verificar tokens con `@nestjs/jwt` y `passport-jwt` mediante estrategias.
- Proteger rutas con guards y renovar el access token a partir del refresh token.
- Analizar críticamente las limitaciones de seguridad del modelo stateless básico.

2. Competencia de la unidad

El estudiante diseña e implementa mecanismos de autenticación basados en tokens para servicios web, aplicando buenas prácticas de seguridad en el manejo de credenciales y evaluando los riesgos asociados a cada decisión de diseño.

3. Marco teórico

3.1 ¿Qué es un JWT?

Un **JSON Web Token (JWT)**, definido en el **RFC 7519**, es un estándar para transmitir información entre dos partes como un objeto JSON compacto y firmado digitalmente. Al estar firmado, su contenido puede verificarse y confiarse sin necesidad de consultar una base de datos en cada petición.

Un JWT se compone de tres partes codificadas en Base64URL y separadas por puntos:

```
header.payload.signature  
  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 .   <- Header  
eyJzdWIiOiJhbmMiLCJlbWVpbCI6Ii4uLiJ9 .   <- Payload  
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQ   <- Signature
```

- **Header** — indica el algoritmo de firma (p. ej. HS256) y el tipo de token.
- **Payload** — contiene los claims (afirmaciones): datos del usuario y metadatos.
- **Signature** — firma HMAC calculada con un secreto; garantiza integridad y autenticidad.

Importante: firmado no es cifrado

El payload de un JWT está codificado, no cifrado. Cualquiera puede leer su contenido decodificando el Base64URL (por ejemplo en jwt.io). La firma solo impide que el token sea modificado sin invalidarlo. Por ello nunca debe colocarse información sensible (contraseñas, datos personales) dentro del payload.

3.2 Claims registrados más comunes

Claim	Significado
sub	Subject: identificador del usuario dueño del token.
iat	Issued At: fecha de emisión (timestamp).
exp	Expiration: fecha de caducidad. Tras ella el token deja de ser válido.
iss / aud	Emisor y audiencia previstos del token (opcionales).

3.3 Access token y refresh token: ¿por qué dos?

Si emitiéramos un único token de larga duración, un robo comprometería la cuenta durante todo ese tiempo. Si fuera de corta duración, el usuario tendría que volver a iniciar sesión constantemente. El esquema de dos tokens resuelve esta tensión:

Característica	Access Token	Refresh Token
Duración	Corta (p. ej. 15 min)	Larga (p. ej. 7 días)
Uso	Acceder a rutas protegidas	Obtener nuevos access tokens
Se envía en	Cada petición protegida	Solo al endpoint /refresh
Riesgo si se filtra	Limitado (caduca pronto)	Alto (vida larga)

3.4 Flujo de autenticación

El ciclo de vida completo de la sesión sigue estos pasos:

1. El usuario se registra o inicia sesión enviando sus credenciales.
2. El servidor verifica las credenciales y responde con un access token y un refresh token.
3. El cliente guarda ambos tokens y envía el access token en la cabecera Authorization: Bearer <token> de cada petición protegida.
4. Cuando el access token expira, el servidor responde 401 Unauthorized.
5. El cliente envía el refresh token al endpoint POST /auth/refresh y recibe un nuevo par de tokens, sin pedir credenciales otra vez.

Modelo stateless

En esta guía el servidor no almacena ninguna sesión: confía únicamente en la firma del token. Esto simplifica el escalado horizontal (cualquier instancia puede validar el token), pero implica que un token válido no puede revocarse antes de su expiración. En la sección 6 analizaremos las consecuencias de seguridad de esta decisión.

4. Requisitos previos

- **Node.js** 20 LTS o superior y npm.
- **PostgreSQL** 14 o superior en ejecución, con un usuario y una base de datos disponibles.
- **NestJS CLI** instalado globalmente (`npm i -g @nestjs/cli`).
- **Un cliente HTTP**: Bruno, Postman, o la extensión REST Client de VS Code (archivos .http).
- **Conocimientos previos**: TypeScript, módulos y proveedores de NestJS, y hashing de contraseñas (GL3).

5. Desarrollo paso a paso

Construiremos la API de autenticación de forma incremental. La estructura final del proyecto será la siguiente:

```
src/
├── app.module.ts
├── main.ts
├── users/
│   ├── user.entity.ts
│   ├── users.module.ts
│   └── users.service.ts
├── auth/
│   ├── auth.module.ts
│   ├── auth.service.ts
│   ├── auth.controller.ts
│   ├── dto/ (register.dto.ts, login.dto.ts)
│   ├── strategies/ (access-token, refresh-token)
│   ├── guards/ (access-token, refresh-token)
│   └── decorators/ (get-current-user)
```

Paso 1 — Crear el proyecto

Generamos un nuevo proyecto NestJS y entramos en la carpeta:

```
nest new inf781-jwt-auth
# Selecciona npm como gestor de paquetes
cd inf781-jwt-auth
```

Paso 2 — Instalar dependencias

Instalamos TypeORM, el driver de PostgreSQL, los paquetes de JWT y Passport, el hasher y los validadores:

```
npm install @nestjs/typeorm typeorm pg \
  @nestjs/jwt @nestjs/passport passport passport-jwt \
  @nestjs/config argon2 class-validator class-transformer

npm install -D @types/passport-jwt
```

Paso 3 — Variables de entorno

Creamos un archivo .env en la raíz del proyecto. Los secretos de access y refresh deben ser distintos y robustos:

Archivo: .env

```
# .env
DATABASE_HOST=localhost
DATABASE_PORT=5432
```

```
DATABASE_USER=postgres
DATABASE_PASSWORD=postgres
DATABASE_NAME=inf781_jwt

JWT_ACCESS_SECRET=cambia-esto-por-un-valor-largo-y-aleatorio
JWT_REFRESH_SECRET=otro-valor-distinto-largo-y-aleatorio
JWT_ACCESS_EXPIRES=15m
JWT_REFRESH_EXPIRES=7d
```

Nunca subas el .env al repositorio

Agrega .env a tu archivo .gitignore. Los secretos en el control de versiones son una de las filtraciones de credenciales más comunes.

Paso 4 — Conexión a PostgreSQL y módulo raíz

Configuramos ConfigModule (global) y la conexión de TypeORM en app.module.ts:

Archivo: src/app.module.ts

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from './users/user.entity';
import { UsersModule } from './users/users.module';
import { AuthModule } from './auth/auth.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    TypeOrmModule.forRoot({
      type: 'postgres',
      host: process.env.DATABASE_HOST,
      port: parseInt(process.env.DATABASE_PORT, 10),
      username: process.env.DATABASE_USER,
      password: process.env.DATABASE_PASSWORD,
      database: process.env.DATABASE_NAME,
      entities: [User],
      synchronize: true, // SOLO en desarrollo
    }),
    UsersModule,
    AuthModule,
  ],
})
export class AppModule {}
```

synchronize: true solo en desarrollo

La opción synchronize crea y modifica tablas automáticamente según las entidades. Es cómoda para aprender, pero en producción puede provocar pérdida de datos. Allí se usan migraciones explícitas.

Paso 5 — Entidad User

Archivo: *src/users/user.entity.ts*

```
import { Column, Entity, PrimaryGeneratedColumn, CreateDateColumn } from 'typeorm';

@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  email: string;

  @Column()
  password: string; // almacena el HASH, nunca el texto plano

  @CreateDateColumn()
  createdAt: Date;
}
```

Paso 6 — Servicio y módulo de usuarios

Archivo: *src/users/users.service.ts*

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { User } from '../user.entity';

@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private readonly repo: Repository<User>,
  ) {}

  create(email: string, password: string): Promise<User> {
    const user = this.repo.create({ email, password });
    return this.repo.save(user);
  }

  findByEmail(email: string): Promise<User | null> {
    return this.repo.findOne({ where: { email } });
  }
}
```

Archivo: *src/users/users.module.ts*

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from '../user.entity';
import { UsersService } from './users.service';
```

```
@Module({
  imports: [TypeOrmModule.forFeature([User])],
  providers: [UsersService],
  exports: [UsersService],
})
export class UsersModule {}
```

Paso 7 — DTOs con validación

Los DTOs definen y validan la forma de las peticiones de entrada con class-validator:

Archivo: *src/auth/dto/register.dto.ts*

```
import { IsEmail, IsString, MinLength } from 'class-validator';

export class RegisterDto {
  @IsEmail()
  email: string;

  @IsString()
  @MinLength(8)
  password: string;
}

// login.dto.ts es idéntico (email + password)
```

Paso 8 — AuthService: registro, login y emisión de tokens

El servicio concentra la lógica: hashea contraseñas con argon2, verifica credenciales y genera el par de tokens firmándolos con secretos y expiraciones distintos.

Archivo: *src/auth/auth.service.ts*

```
import { ForbiddenException, Injectable, UnauthorizedException } from '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import { ConfigService } from '@nestjs/config';
import * as argon2 from 'argon2';
import { UsersService } from '../users/users.service';
import { RegisterDto } from '../dto/register.dto';
import { LoginDto } from '../dto/login.dto';

@Injectable()
export class AuthService {
  constructor(
    private readonly users: UsersService,
    private readonly jwt: JwtService,
    private readonly config: ConfigService,
  ) {}

  async register(dto: RegisterDto) {
    const existing = await this.users.findByEmail(dto.email);
    if (existing) throw new ForbiddenException('El correo ya está registrado');
    const hash = await argon2.hash(dto.password);
```



```

    const user = await this.users.create(dto.email, hash);
    return this.getTokens(user.id, user.email);
  }

  async login(dto: LoginDto) {
    const user = await this.users.findByEmail(dto.email);
    if (!user) throw new UnauthorizedException('Credenciales inválidas');
    const ok = await argon2.verify(user.password, dto.password);
    if (!ok) throw new UnauthorizedException('Credenciales inválidas');
    return this.getTokens(user.id, user.email);
  }

  // Versión básica stateless: confiamos en la firma del refresh token.
  async refreshTokens(userId: string, email: string) {
    return this.getTokens(userId, email);
  }

  private async getTokens(sub: string, email: string) {
    const payload = { sub, email };
    const [accessToken, refreshToken] = await Promise.all([
      this.jwt.signAsync(payload, {
        secret: this.config.get('JWT_ACCESS_SECRET'),
        expiresIn: this.config.get('JWT_ACCESS_EXPIRES'),
      }),
      this.jwt.signAsync(payload, {
        secret: this.config.get('JWT_REFRESH_SECRET'),
        expiresIn: this.config.get('JWT_REFRESH_EXPIRES'),
      }),
    ]);
    return { accessToken, refreshToken };
  }
}

```

Paso 9 — Estrategia del access token

La estrategia "jwt" extrae el token del header Bearer y valida su firma con el secreto de acceso. Lo que devuelve validate() se adjunta a request.user.

Archivo: `src/auth/strategies/access-token.strategy.ts`

```

import { Injectable } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';

type JwtPayload = { sub: string; email: string };

@Injectable()
export class AccessTokenStrategy extends PassportStrategy(Strategy, 'jwt') {
  constructor(config: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      secretOrKey: config.get('JWT_ACCESS_SECRET'),
    });
  }
}

```

```
}

validate(payload: JwtPayload) {
  return payload; // -> request.user
}
}
```

Paso 10 — Estrategia del refresh token

La estrategia "jwt-refresh" valida con el secreto de refresh y, gracias a `passReqToCallback`, expone el propio refresh token en `request.user` para usos posteriores.

Archivo: `src/auth/strategies/refresh-token.strategy.ts`

```
import { Injectable } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { PassportStrategy } from '@nestjs/passport';
import { ExtractJwt, Strategy } from 'passport-jwt';
import { Request } from 'express';

type JwtPayload = { sub: string; email: string };

@Injectable()
export class RefreshTokenStrategy extends PassportStrategy(
  Strategy, 'jwt-refresh',
) {
  constructor(config: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      secretOrKey: config.get('JWT_REFRESH_SECRET'),
      passReqToCallback: true,
    });
  }

  validate(req: Request, payload: JwtPayload) {
    const refreshToken = req
      .get('Authorization')?.replace('Bearer ', '').trim();
    return { ...payload, refreshToken };
  }
}
```

Paso 11 — Guards y decorador

Los guards aplican cada estrategia a las rutas. El decorador `@GetCurrentUser` facilita leer datos del usuario autenticado.

```
// guards/access-token.guard.ts
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

@Injectable()
export class AccessTokenGuard extends AuthGuard('jwt') {}
```

```
// guards/refresh-token.guard.ts
@Injectable()
export class RefreshTokenGuard extends AuthGuard('jwt-refresh') {}
```

Archivo: *src/auth/decorators/get-current-user.decorator.ts*

```
import { createParamDecorator, ExecutionContext } from '@nestjs/common';

export const GetCurrentUser = createParamDecorator(
  (data: string | undefined, ctx: ExecutionContext) => {
    const req = ctx.switchToHttp().getRequest();
    return data ? req.user[data] : req.user;
  },
);
```

Paso 12 — AuthModule y AuthController

Archivo: *src/auth/auth.module.ts*

```
import { Module } from '@nestjs/common';
import { JwtModule } from '@nestjs/jwt';
import { PassportModule } from '@nestjs/passport';
import { UsersModule } from '../users/users.module';
import { AuthService } from './auth.service';
import { AuthController } from './auth.controller';
import { AccessTokenStrategy } from './strategies/access-token.strategy';
import { RefreshTokenStrategy } from './strategies/refresh-token.strategy';

@Module({
  imports: [PassportModule, JwtModule.register({}), UsersModule],
  controllers: [AuthController],
  providers: [AuthService, AccessTokenStrategy, RefreshTokenStrategy],
})
export class AuthModule {}
```

Archivo: *src/auth/auth.controller.ts*

```
import { Body, Controller, Get, HttpStatusCode, HttpStatus, Post, UseGuards } from '@nestjs/common';
import { AuthService } from './auth.service';
import { RegisterDto } from './dto/register.dto';
import { LoginDto } from './dto/login.dto';
import { AccessTokenGuard } from './guards/access-token.guard';
import { RefreshTokenGuard } from './guards/refresh-token.guard';
import { GetCurrentUser } from './decorators/get-current-user.decorator';

@Controller('auth')
export class AuthController {
  constructor(private readonly auth: AuthService) {}

  @Post('register')
  register(@Body() dto: RegisterDto) {
    return this.auth.register(dto);
  }
}
```

```
}

@HttpCode(HttpStatus.OK)
@Post('login')
login(@Body() dto: LoginDto) {
  return this.auth.login(dto);
}

@UseGuards(RefreshTokenGuard)
@HttpCode(HttpStatus.OK)
@Post('refresh')
refresh(@GetCurrentUser('sub') id: string,
        @GetCurrentUser('email') email: string) {
  return this.auth.refreshTokens(id, email);
}

@UseGuards(AccessTokenGuard)
@Get('me')
me(@GetCurrentUser() user: { sub: string; email: string }) {
  return user;
}
}
```

Paso 13 — Habilitar validación global

En main.ts activamos el ValidationPipe global para que los DTOs se validen automáticamente:

Archivo: src/main.ts

```
import { NestFactory } from '@nestjs/core';
import { ValidationPipe } from '@nestjs/common';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.useGlobalPipes(new ValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
  }));
  await app.listen(3000);
}
bootstrap();
```

Paso 14 — Probar el flujo completo

Arranca el servidor con `npm run start:dev` y ejecuta las siguientes peticiones en orden (ejemplos con la extensión REST Client de VS Code; equivalen a peticiones de Postman o Bruno):

Archivo: requests.http

```
### 1) Registro
POST http://localhost:3000/auth/register
Content-Type: application/json
```

```
{ "email": "ana@uatf.edu.bo", "password": "Segura123" }
```

```
### 2) Login -> copia accessToken y refreshToken
```

```
POST http://localhost:3000/auth/login
```

```
Content-Type: application/json
```

```
{ "email": "ana@uatf.edu.bo", "password": "Segura123" }
```

```
### 3) Ruta protegida (usa el access token)
```

```
GET http://localhost:3000/auth/me
```

```
Authorization: Bearer <accessToken>
```

```
### 4) Renovar tokens (usa el refresh token)
```

```
POST http://localhost:3000/auth/refresh
```

```
Authorization: Bearer <refreshToken>
```

Verifica el comportamiento esperado

Llama a /auth/me con un access token caducado o inválido: debes recibir 401 Unauthorized. Luego renueva con /auth/refresh y repite la petición con el nuevo access token: ahora debe responder 200. Ese es el ciclo que justifica el esquema de dos tokens.

6. Análisis de seguridad

Por tratarse de una materia de seguridad, es esencial reconocer qué protege y qué NO protege la implementación que acabamos de construir.

6.1 Qué garantiza el modelo

- Integridad y autenticidad: un token no puede modificarse sin invalidar la firma.
- Contraseñas protegidas: se almacena el hash argon2, nunca el texto plano.
- Exposición acotada del access token gracias a su corta duración.

6.2 Limitaciones de esta versión básica

Riesgos del modelo stateless sin persistencia

No hay revocación. Como no guardamos los refresh tokens, no es posible invalidar uno antes de su expiración. Un "logout" real no puede cerrar la sesión del lado del servidor.

Robo de refresh token. Si un atacante obtiene el refresh token, puede generar nuevos access tokens hasta que aquel caduque (días), sin que podamos detenerlo.

Almacenamiento en el cliente. Guardar tokens en localStorage los expone a ataques XSS. El header Bearer es práctico para aprender, pero en producción el refresh suele ir en una cookie httpOnly.

6.3 Buenas prácticas mínimas

- Usar HTTPS siempre: en texto plano, cualquier intermediario puede leer los tokens.
- Secretos largos, aleatorios y distintos para access y refresh; gestionarlos por variables de entorno.
- Access tokens de vida corta (5–15 min) y refresh tokens con vida razonable (días, no meses).
- No incluir datos sensibles en el payload (recuerda: está codificado, no cifrado).

Hacia dónde sigue (próxima guía)

El siguiente paso natural es convertir el esquema en stateful para el refresh: persistir un hash del refresh token en la base de datos, aplicar rotación (emitir un refresh nuevo en cada uso e invalidar el anterior) y permitir revocación real en el logout. Esto habilita detección de reutilización de tokens robados.

7. Bibliografía y referencias

NestJS. (2026). *Authentication. Documentación oficial* — docs.nestjs.com/security/authentication

Internet Engineering Task Force. (2015). *RFC 7519: JSON Web Token (JWT)*.

Auth0. *JWT.io — Introduction to JSON Web Tokens*. jwt.io/introduction

Passport. *passport-jwt — JWT authentication strategy*. github.com/mikenicholson/passport-jwt

P-H-C. *argon2 — Password hashing library for Node.js*. npmjs.com/package/argon2